Technical Interview Script: GCP Data Engineer (5 Years Experience)

Candidate Focus Areas (Based on Resume): GCP (BigQuery, Composer, Dataflow, Pub/Sub), Python, SQL, Data Masking/Security, ETL Architecture (Staging, History, Golden Layers), and Performance Optimization.

Section 1: Core GCP Data Services & Optimization (BigQuery Focus) (8 Questions)

Q1: BigQuery Partitioning, Clustering, and Performance

Question: Your resume mentions optimizing BigQuery queries, table partitioning, and clustering strategies, cutting query costs by 35% and improving performance by 40%. Describe a specific scenario where you utilized both **time-unit partitioning** and **clustering** on the same table. Explain *why* that combination was necessary and how you measured the 40% performance gain.

Expected Answer: I would use partitioning on an **ingestion or event timestamp** column (e.g., `event_date`) to eliminate scanning entire days/months. I would then use clustering on a **high-cardinality filter column** (e.g., `customer_id` or `country_code`). The combination is necessary because partitioning handles time filtering, and clustering handles selective querying *within* those partitions, saving I/O. The performance gain (40%) was measured by comparing the **slot time used** and the **bytes processed** in the BigQuery query metadata before and after the optimization, targeting high-impact daily analytical queries.

Q2: BigQuery Cost Management

Question: Beyond pruning and partitioning/clustering, what is one advanced BigQuery technique you've used to manage query costs, specifically when dealing with complex joins across very large tables?

Expected Answer: I primarily use **Materialized Views** or **Clustering on Join Keys**. Materialized views pre-compute the results of frequently run, complex joins, allowing subsequent queries to read the smaller, pre-processed view instead of re-scanning the massive base tables. This shifts the cost from high-volume interactive queries to a controlled, scheduled update cost.

Q3: BigQuery Data Loading Strategy

Question: When streaming high-volume data via Pub/Sub to BigQuery, what are the primary pros and cons of using the **BigQuery Storage Write API** compared to the older **Streaming Inserts API**?

Expected Answer: I would prioritize the **BigQuery Storage Write API** for reliability.

- **Pros (Storage Write API):** It supports **exactly-once delivery semantics**, uses a commit-based transaction model, and has higher throughput limits. This is crucial for data integrity.
- **Cons (Streaming Inserts API):** It only guarantees at-least-once delivery, leading to potential duplicates that must be handled downstream, increasing ETL complexity.

Q4: Handling Nested/Repeated Fields in BigQuery

Question: When ingesting semi-structured data (like JSON) into BigQuery, what best practice do you follow regarding nested fields, and how do you query data stored in a `REPEATED RECORD` structure?

Expected Answer: I would maintain the nested structure by mapping the JSON array to a BigQuery `REPEATED RECORD` (array of structs). This preserves semantic meaning and avoids denormalization overhead. To query the nested data, I use the `UNNEST` operator in the `FROM` clause, which flattens the array into separate rows temporarily, allowing me to join or filter on the nested fields.

Q5: Choosing the Right GCS Storage Class

Question: If you are storing raw event logs in Cloud Storage that are accessed only once per day by a Dataflow pipeline, and then rarely again for audit, which storage class would you select (`Standard` , `Nearline` , `Coldline`), and why?

Expected Answer: I would select the **Nearline** storage class. Since the data is accessed roughly once a day, the retrieval fees of Nearline are minimal, providing a better cost-saving balance compared to Standard (which has high storage cost) and Coldline/Archive (which have higher retrieval fees and minimum retention periods).

Q6: Data Quality and Reconciliation

Question: Detail your data validation and reconciliation framework used to handle record count checks and anomaly detection. Where in your staging→history→golden architecture do you place the most stringent checks?

Expected Answer: The most stringent checks are placed at the **History and Golden Layers**.

- **Staging:** Basic checks (schema validation, null checks on key fields).
- **History/Golden: Reconciliation** (Record Count, Sum/Average checks between the staging and history layers). **Anomaly Detection** (Outlier analysis, checking for schema drift). The **Golden Layer** has the final, business-logic driven data quality checks enforced.

Q7: Python Data Handling

Question: Describe a time when you used Python scripting to handle a data ingestion or transformation problem that was too complex for standard SQL or Airflow operators alone.

Expected Answer: I used Python to handle complex data transformation when dealing with dynamic, semi-structured data sources, specifically for **custom schema flattening and standardization**. Instead of complex SQL or slow Dataflow, I used Python's native `json` library and custom functions to parse and validate inconsistent JSON files from GCS, standardize column names, and transform complex arrays into a flat structure before loading into BigQuery.

Q8: Data Masking and Compliance

Question: Explain the difference between **data masking** and **format-preserving encryption (FPE)** in the context of PII compliance. When would you use FPE over simple hashing for sensitive fields?

Expected Answer:

- **Data Masking:** Replaces sensitive data with realistic, fake data (e.g., replacing a real SSN with a fake, random SSN). It is often irreversible.

- **FPE (Format-Preserving Encryption):** Encrypts the data while maintaining the original data's length and format (e.g., an 8-digit SSN encrypts to a different 8-digit SSN).
- **Use Case for FPE:** I use FPE when the sensitive field **must be joined or indexed downstream** (e.g., customer IDs, email addresses used as join keys). FPE allows the masked value to be consistently maintained for joins, enabling data lineage and traceability without exposing the raw PII.

Section 2: Data Orchestration and Real-Time (Cloud Composer/Airflow & Pub/Sub) (8 Questions)

Q9: Cloud Composer (Airflow) DAG Architecture

Question: When designing DAGs in Cloud Composer for production, how do you handle **idempotency** and ensure that a failed task, when retried, doesn't duplicate data in the target BigQuery table?

Expected Answer: Idempotency is critical. I ensure this by implementing **delete-then-insert** or **merge** operations in the downstream BigQuery task. Instead of direct append:

1. **Intermediate Table:** Write data from the failed task to a temporary staging table (`_temp_`).
2. **Merge/Replace:** Use a `BigQueryInsertJobOperator` with a SQL MERGE statement or a `WRITE_TRUNCATE` disposition on a partitioned table to atomically replace the target data based on the unique key/date, ensuring no duplicates.

Q10: Airflow Sensor vs. Deferrable Operators

Question: You need to wait for a large file to land in a Cloud Storage bucket before triggering a Dataflow job. Would you use the legacy `GCSObjectSensor` or a **Deferrable Operator (AIP-40)**, and why?

Expected Answer: I would use a **Deferrable Operator** (or a **Serverless function triggered by GCS event**).

- **Reason:** Legacy sensors occupy a worker slot for the entire wait time, consuming resources and costing money. Deferrable Operators release the worker and move the task to a lightweight trigger process, saving significant costs and increasing worker availability for actual computation.

Q11: Real-Time Ingestion with Pub/Sub

Question: Describe the typical GCP pipeline you would build to handle high-throughput **streaming data** from Pub/Sub into BigQuery for near-real-time analytics, avoiding the use of Dataflow templates.

Expected Answer: The standard, robust pipeline is: **Source Application → Pub/Sub → Cloud Dataflow (Custom Beam Pipeline) → BigQuery.**

- **Pub/Sub:** Handles durability, buffering, and decoupling the source from the processing engine.
- **Dataflow:** Serves as the processing layer. I use a custom Apache Beam pipeline for transformation, windowing, and managing state (e.g., 5-minute running aggregates).
- **BigQuery:** Used as the sink for streaming inserts via the optimized BigQuery I/O connector.

Q12: Dataflow Auto-Scaling

Question: How does Dataflow's auto-scaling differ from standard VM instance scaling, and what two metrics do you monitor in Dataflow to detect and manage worker backlogs?

Expected Answer: Dataflow's auto-scaling is **pipeline-aware and work-based**. It scales based on the current volume of unprocessed data (the *backlog*) and processing saturation, not just general CPU usage. The two key metrics to monitor for backlogs are:

1. **System Latency:** The time it takes for a message to enter the pipeline and reach the sink. A sharp rise indicates a bottleneck.
2. **Current VCPU Utilization:** If VCPU usage is maxed out and latency is rising, it means workers are saturated and need to be scaled up.

Q13: Handling Sensitive Credentials

Question: Where do you securely store sensitive credentials (like database passwords or API keys) used by your Python scripts and Airflow DAGs running in Cloud Composer?

Expected Answer: I store sensitive credentials in **Google Cloud Secret Manager**. Airflow/Python scripts running within Cloud Composer can securely access these secrets using the standard GCP SDK library, which integrates seamlessly with Secret Manager and leverages the service account permissions of the Composer cluster. I never hardcode credentials in DAG files.

Q14: CI/CD for Airflow

Question: You use Git/Bitbucket and Cloud Build for CI/CD. Outline the steps in your Cloud Build pipeline for deploying a new Airflow DAG version to Cloud Composer.

Expected Answer:

1. **Trigger:** A pull request merge to the `main` branch triggers the Cloud Build pipeline.
2. **Lint/Test:** Cloud Build executes linting and unit tests on the Python DAG file.
3. **Deployment:** If tests pass, Cloud Build uses the `gcloud composer environments storage dags import` command (or the GCS transfer) to upload the new DAG file to the correct GCS bucket associated with the Cloud Composer environment.
4. **Verification:** The DAG is verified by the Composer environment, becoming active within minutes.

Q15: Error Handling in DAGs

Question: How do you implement robust retry logic and error handling in an Airflow DAG to prevent transient failures from causing major pipeline breaks?

Expected Answer: I implement the following:

- `default_args` : Set global `retries` (e.g., 3) and `retry_delay` (e.g., 5 minutes) for all tasks.
- `on_failure_callback` : Define a callback function that sends an alert (via Slack, PagerDuty, or Pub/Sub) upon final task failure.
- **Branching/SLA:** Use the `BranchPythonOperator` or `ExternalTaskSensor` to create failure paths that clean up resources or send alerts *before* the entire DAG fails, and set an `SLA` for critical tasks.

Q16: Data Modeling Layers

Question: Briefly define the purpose of the three main data layers you've worked with: **Staging, History, and Golden**.

Expected Answer:

1. **Staging Layer:** Temporary storage for raw, minimally processed data immediately after ingestion. Used for light cleaning and data validation.
2. **History Layer (or Raw/Bronze):** Immutable, exact copy of all source data, including every historical change. Used for audit, lineage, and re-processing.
3. **Golden Layer (or Mart/Silver/Gold):** Highly curated, transformed, aggregated, and modeled data that is optimized for business analytics and reporting.

Section 3: Scenario & Behavioral Questions (6 Questions)

Q17: Data Quality Incident Management

Question: You receive an alert that the record count from an ETL pipeline dropped to zero this morning, but the upstream source system (a remote database) reports a normal status. Walk me through your immediate steps to diagnose and resolve this incident.

Expected Answer:

1. **Acknowledge and Check Monitoring:** Immediately check Airflow/Cloud Composer logs for the failed task. Look at Dataflow/Dataproc logs for the specific pipeline execution. Check Pub/Sub or GCS for evidence of incoming data (was data generated?).
2. **Isolate the Failure Point:** Compare the record count between the source query log and the staging layer. If the source query executed successfully but staged zero records, the failure is in the **ingestion logic** (e.g., a dynamic date parameter failed, or the schema changed). If the source query itself failed, the issue is external (network, permissions, or source system change).
3. **Resolution:** If the issue is a transient error, rely on the pipeline's retry logic. If it's a code/schema change, rollback the DAG version via Git/Cloud Build, fix the code, and manually re-run the affected historical load.

Q18: Architectural Compromise

Question: Your product manager insists on using Cloud SQL for a high-volume event stream sink because the analysts only know SQL, even though you recommend BigQuery for scale. How do you respond, and what is your proposed compromise architecture?

Expected Answer: I would first present the **data limits** of Cloud SQL (transaction volume, write IOPS, cost) compared to BigQuery's native scale, explaining that the suggested architecture will fail under high load.

- **Compromise:** I would propose a **two-tier architecture**:
    1. Ingest all raw data into **BigQuery** (the high-scale, cost-effective truth).
    2. Use **federated queries** from BigQuery into a small, aggregated view in Cloud SQL, or use a tool like **Cloud Dataflow** to write small, aggregated, reporting-only tables from BigQuery back into Cloud SQL. This meets the analyst's requirement while maintaining system integrity and scalability.

Q19: Handling Data Skew

Question: During a large-scale PySpark job running on Dataproc, you notice one worker is running at 100% CPU while others are idle, causing the job to crawl. What is this phenomenon called, and what is the typical technique you use to mitigate it?

Expected Answer: This is called **Data Skew**. It occurs when a highly unequal distribution of data (e.g., 80% of records share the same `user_id` which becomes the join key) causes one worker to be assigned most of the computational load.

- **Mitigation (Salting):** I would use a **Salting technique**. I would append a random prefix/suffix (a "salt") to the skewed key (e.g., `user_id_1`, `user_id_2`). I would then join on the salted key, forcing the highly skewed data to be distributed across multiple workers, followed by de-salting the keys later if necessary.

Q20: Technical Debt Management

Question: A pipeline you built two years ago is now costing 30% more than it should due to outdated operators and unoptimized BigQuery logic. How do you prioritize refactoring this technical debt alongside new feature development?

Expected Answer: I use a data-driven approach:

1. **Quantify Impact:** I would quantify the refactoring effort (estimated development time) vs. the continuous cost saving (30% monthly cost reduction).
2. **Prioritization:** I treat the refactoring as an **SLA/Cost Optimization task** rather than a feature. If the ROI (cost savings vs. time spent) is high and the solution is low-risk, I dedicate a small percentage (e.g., 20%) of sprint capacity to it.
3. **Phased Rollout:** Refactor the pipeline in stages, using A/B testing (running the old and new version in parallel and comparing output and metrics) before cutting over fully.

Q21: Learning New GCP Services

Question: If you had to rapidly integrate a new, unfamiliar GCP service (like Vertex AI Feature Store) into an existing pipeline, what is your standard learning and integration process?

Expected Answer:

1. **Documentation Focus:** I start with the **official Google Cloud documentation** to understand the service's API, security model (IAM roles), and core concepts.
2. **POC (Proof of Concept):** I build a minimal, isolated Python script or a very simple, non-production DAG to successfully connect to the service, authenticate, and perform the basic CRUD operations.
3. **Integration:** I then introduce the necessary client libraries and service account permissions into the target Dataflow/Composer environment, followed by integrating the logic into the development environment pipeline.

Q22: iOS Background (Strength)

Question: Your background includes 2 years of iOS development. How does that experience in application engineering make you a stronger GCP Data Engineer, specifically regarding data integrity and system design?

Expected Answer: My iOS background provides a strong appreciation for the **application ecosystem**.

1. **Data Integrity:** I understand the upstream **source of truth** and the real-time constraints of application databases (CoreData) and APIs (REST APIs). This allows me to design pipeline ingestion APIs that minimize load on the production application and anticipate data integrity issues at the source.
2. **System Design:** Experience with MVC/MVVM architectures translates directly to layered data modeling (Staging/Golden). Furthermore, focusing on performance optimization in Swift transfers